Benjamin Morin
Independent Security Consultant

Case Study: Architectural Vulnerability in a Cross-Chain Liquidity Protocol

1. Executive Summary

This report details the findings of an independent security assessment conducted on a prominent cross-chain liquidity protocol. The assessment uncovered a critical, systemic vulnerability deeply embedded within the protocol's core pool contracts. This severe flaw stemmed from a dangerous combination: an unbounded parameter in a highly privileged function, coupled with a manipulable mathematical calculation in another. This chained exploit, if leveraged by a malicious actor controlling privileged roles, would enable the complete circumvention of the protocol's established withdrawal limits, leading to the full drainage of all token balances across any pool. This scenario would inevitably result in total protocol insolvency. At the time of discovery, this architectural design failure represented a direct and catastrophic threat to the protocol's entire **~$252 million** in Total Value Locked (TVL).

2. Methodology

The assessment focused meticulously on identifying architectural failures within the on-chain access control mechanisms, specifically investigating how privileged roles could potentially manipulate the protocol's internal accounting state. The methodology employed a rigorous, multi-faceted approach:

- **Privileged Role Analysis:** A comprehensive and in-depth review was conducted on the capabilities and permissions granted to all privileged roles (referred to herein as 'Planner' and 'Treasurer'). The objective was to thoroughly understand their potential for malicious state manipulation and unauthorized actions within the protocol.
- **Static Code Analysis:** A meticulous manual audit of the core `StargatePool.sol` smart contract was performed. This analysis aimed to identify functions that lacked critical input validation, relied on internal state variables susceptible to influence by privileged actors, or exhibited any other potential vulnerabilities.
- **Exploit Chain Development:** A core emphasis of the analysis was to move beyond the identification of isolated, single-function bugs. Instead, the focus was on identifying a logical chain of exploits. This involved theorizing how a specific flaw in one privileged

function could be strategically exploited to create the necessary conditions to then exploit a second, seemingly unrelated function, thus achieving a cascading effect.
- **Proof-of-Concept (PoC) Formulation:** Based on the theoretical exploit chains, a detailed, step-by-step exploit path was meticulously designed. This PoC demonstrated precisely how an attacker, by first corrupting the pool's internal accounting state through one privileged function, could subsequently use a separate function to bypass all withdrawal caps and ultimately drain all user-deposited funds.

3. The Critical Finding: Chained Architectural Flaw

The vulnerability discovered was a sophisticated, two-part exploit chain that ingeniously leveraged a fundamental flaw in the protocol's administrative controls to corrupt its financial accounting system.

- **Part A: Unbounded State Manipulation:** A critical flaw was identified in the privileged function `setDeficitOffset()`, which was intended for high-level pool management. Crucially, this function completely lacked any form of input validation. This egregious oversight allowed a malicious Planner to arbitrarily set a key accounting variable, `deficitOffsetSD`, to an astronomically large number. This deliberate action was designed to corrupt the protocol's internal accounting, thereby forcing the main pool balance variable (`poolBalanceSD`) to become zero or near-zero, regardless of the actual funds present.
- **Part B: Exploitation of Flawed Calculation:** A distinct and separate function, `recoverToken()`, utilized by the Treasurer to withdraw surplus funds, contained a critical mathematical flaw. This function calculated its withdrawal limit based on the flawed assumption that the `poolBalanceSD` variable accurately reflected the true balance of the pool. With the pool's state already severely corrupted by the malicious action performed in Part A, the `recoverToken()` function's withdrawal limit calculation was rendered entirely useless. Consequently, the effective withdrawal cap was set to the entire token balance of the pool, allowing for unlimited withdrawals.

An attacker with control over both the Planner and Treasurer roles could therefore first corrupt the protocol's internal state using `setDeficitOffset()`, and then exploit the `recoverToken()` function to withdraw all user funds, leading to the complete insolvency of the protocol.

4. Business Impact & Risk Assessment

This finding represented an architectural failure of the absolute highest severity. The protocol's entire security model was built upon the critically flawed assumption that privileged roles would

act benevolently, yet it provided no on-chain safeguards whatsoever to prevent them from acting maliciously. The potential impact of exploitation would have been catastrophic:

- **Total Loss of User Funds:** The vulnerability directly enabled the outright theft of the entire **~$252 million** in user-deposited funds held across all Stargate V2 pools. This would represent an unprecedented financial loss for the protocol's users.
- **Complete Protocol Insolvency:** The draining of even a single major pool would inevitably trigger a widespread bank run and an irreversible collapse of confidence among users and liquidity providers. This would render the entire protocol fundamentally insolvent and utterly worthless.
- **Irreparable Reputational Damage:** An exploit of this nature, stemming from such a fundamental and glaring design flaw, would permanently destroy user trust and irrevocably tarnish the brand's reputation within the highly competitive and trust-dependent Decentralized Finance (DeFi) ecosystem.

5. Conclusion

This engagement unequivocally demonstrates the critical and urgent need for a "zero-trust" approach to smart contract design, even when considering ostensibly privileged roles. It emphatically proves that robust security is not merely about preventing external attacks, but fundamentally about implementing stringent internal controls, strict input validation, and comprehensive on-chain safeguards. These measures are crucial to protect against the failure or malicious action of any single component, especially those with elevated privileges. The timely discovery of this chained, architectural flaw prevented a catastrophic, nine-figure financial loss and vividly highlighted the immense value and necessity of deep, systemic security analysis in the blockchain space.